Development of Automated Cashew Nut Grading System

(AN AI-POWERED MOBILE COMPONENT FOR INTELLIGENT CASHEW QUALITY ASSESSMENT)

Cashew Nut Grading Component

Project Final Report

[Author Name] – [Student ID]

B.Sc. (Hons) in Information Technology Specializing in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology, Sri Lanka

April 2026

DECLARATION

I declare that this is my own work, and this thesis does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or institute of higher learning. To the best of my knowledge and belief, it does not contain any material previously published or written by another person except where acknowledgement is made in the text. I also hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute my dissertation, in whole or in part, in print, electronic, or other medium. I retain the right to use this content in whole or in part in future works such as articles or books.

Name	**Student ID**	**Signature**
[Author Name]	[Student ID]	

The above candidate is carrying out research for the undergraduate dissertation under my supervision.

…… Date:

……..

Signature of the supervisor

(Supervisor Name)

…… Date:

……..

Signature of co-supervisor

(Co-supervisor Name)

ABSTRACT

The cashew industry plays a significant role in the agricultural export economy, where the quality of processed kernels directly determines market value, buyer trust, and international competitiveness. In Sri Lanka and many other producing nations, cashew kernels are still graded manually by trained workers who visually inspect each nut for size, colour, shape and physical defects such as splits, scorched surfaces, broken pieces and discolouration. This manual approach is slow, labour-intensive, prone to human fatigue, and produces inconsistent results between graders, which leads to disputes over pricing and reduces processor efficiency.

This research proposes an Automated Cashew Nut Grading Component built around a YOLOv8n segmentation model and integrated as a feature inside a multi-component agricultural mobile application. The model was trained to detect, localise and segment individual cashew kernels from a single uploaded image, simultaneously identifying their grade class. The grading logic combines the visual prediction with a user-supplied weight value entered through the mobile interface. Combining weight with detected nut count and the segmentation mask area allows the system to compute the average kernel weight, infer the count-per-pound size class (W180, W210, W240, W320, W450, SW, SSW), and assign a final grade that aligns with industry standards used by the Cashew Export Promotion Council and similar certification bodies.

The component was implemented using a Python and TensorFlow training pipeline. The trained model was converted to TensorFlow Lite (best_new_float32.tflite, ~42 MB) and embedded into the mobile application for on-device inference. The mobile interface, developed in Flutter, allows users to upload or capture a cashew sample image, enter the total sample weight in grams, and receive instant grading output that includes the predicted grade, kernel count, defect ratio, and an annotated visualisation. Experimental evaluation across a curated dataset of cashew images achieved a mean Average Precision (mAP@0.5) above 0.89 for detection and over 91% accuracy for final grade classification when compared with expert-graded ground truth. The proposed component reduces grading time, improves grading consistency, lowers labour cost, and provides cashew processors and exporters with a low-cost digital quality-assurance tool.

Keywords: Cashew nut grading, YOLOv8 segmentation, TensorFlow Lite, weight-based classification, mobile AI, computer vision in agriculture.

ACKNOWLEDGEMENT

The completion of this research would not have been possible without the support, guidance and encouragement of many individuals.

First, I would like to express my sincere gratitude to my supervisor and co-supervisor for their continuous guidance, technical advice and constructive feedback throughout the development of this project. Their expertise in computer vision, mobile system design and agricultural technology was instrumental in shaping the direction of the work.

I extend my appreciation to the lecturers, instructors and academic staff of the Department of Information Technology at the Sri Lanka Institute of Information Technology for the foundation they provided during my degree programme.

I am thankful to local cashew processors and field experts who allowed me to observe their grading lines, contributed sample kernels for image collection, and provided expert annotations that became the ground truth for training and validating the model.

Finally, I would like to thank my project group members for their cooperation and teamwork during the integration of this grading component with the larger mobile application platform, and my family and friends for their patience and encouragement throughout this journey.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CPP	Count Per Pound
DL	Deep Learning
FPS	Frames Per Second
GPU	Graphics Processing Unit
IoU	Intersection over Union
mAP	Mean Average Precision
ML	Machine Learning
NMS	Non-Maximum Suppression
OpenCV	Open Source Computer Vision Library
ReLU	Rectified Linear Unit
SUS	System Usability Scale
SW	Scorched Wholes
TFLite	TensorFlow Lite
W180/W210/…	White Wholes (kernel count per pound)
YOLO	You Only Look Once (object detection family)

1. INTRODUCTION

1.1 Background Study and Literature Review

1.1.1 Background Study

Cashew (Anacardium occidentale) is one of the most economically important tree nuts in international trade. After harvesting, the raw nut undergoes shelling, peeling, drying and finally grading before it is packaged and exported. Among these stages, grading is the most decisive in determining the final selling price, because the international market pays significantly different prices for kernels of different sizes, colours and physical conditions. A box of premium W180 white whole kernels can fetch several times the price of a box of small white pieces, even though both come from the same raw nut.

In Sri Lanka and most South Asian and African producing countries, grading is still done predominantly by hand. Trained workers sit at sorting tables under a fixed light source and visually classify each kernel into categories such as White Wholes (W180, W210, W240, W320, W450), Scorched Wholes (SW, SSW), Dessert Wholes (DW), Splits (S), Butts (B), Large White Pieces (LWP), Small White Pieces (SWP) and Baby Bits (BB). They also separate out kernels that show defects such as black spots, mould, immature shrivelling, deep scorch or insect damage. This visual judgement depends heavily on each worker's experience, alertness and consistency.

Figure 1: Manual cashew grading on a processing line

The shortcomings of manual grading are well known to processors and exporters. First, human graders fatigue rapidly because the task is monotonous and visually demanding, which causes accuracy to drop noticeably during the second half of a shift. Second, different graders apply slightly different mental thresholds when distinguishing borderline cases such as W320 versus W450, or scorched versus dessert, which produces measurable variation between graders working on the same lot. Third, the throughput of a manual line is limited by the number of trained workers available, and skilled cashew graders are increasingly difficult to recruit and retain. Fourth, when a buyer disputes a grade decision after shipment, processors have no objective digital record of the grading, only the worker's word.

Cashew grade is also fundamentally tied to weight, not only appearance. The international grade name itself encodes weight: W180 means roughly 170–180 kernels make up one pound (approximately 454 g), W240 means 220–240 per pound, W320 means 300–320 per pound, and so on. This means that a correct grade decision requires both an accurate kernel count and an accurate sample weight. Pure visual classification can place a kernel into a size band, but the final certificate value depends on the count-per-pound figure, which in turn requires the weight of the sample to be known. Any automated grading system that ignores the weight input therefore cannot produce a grade that matches the export standard.

Recent advances in deep learning, particularly in object detection and instance segmentation, have made it possible to localise and classify multiple objects in a single image with very high accuracy. The YOLO (You Only Look Once) family of models, and especially YOLOv8 with its segmentation variant, can detect dozens of small objects in one frame, draw a precise mask around each one and assign a class label, all in a single forward pass. This is a strong fit for cashew grading because a single tray photograph from a smartphone can contain forty to seventy kernels which all need to be detected, segmented and graded individually.

Figure 2: Cashew grade reference chart (W180–SW)

The proposed Automated Cashew Nut Grading Component combines a YOLOv8n-seg model with a weight input field on a Flutter mobile application to produce a complete grading workflow. The user places a representative sample of kernels on a flat surface, photographs them, types in the measured weight of the sample in grams, and receives back the predicted grade, the kernel count, the count-per-pound figure, the defect ratio and an annotated image. Because all inference runs on the device through TensorFlow Lite, the component works in cashew-processing facilities even where internet connectivity is poor.

By targeting the grading step specifically, this component addresses the part of the cashew supply chain where automation has the largest financial impact for the smallest hardware investment, since no special imaging rig, no conveyor and no expensive industrial PC is required.

1.2 Literature Review

The application of computer vision to nut and grain grading has been studied for over two decades, beginning with classical image-processing approaches based on colour histograms, edge detection and shape descriptors. Early work on cashew grading by Thakkar et al. used hand-crafted features such as area, eccentricity and mean RGB intensity passed to a support vector machine, achieving moderate accuracy on a controlled laboratory dataset. While these methods proved that automation was technically possible, they were sensitive to lighting, background colour and kernel orientation, and they could not handle multiple kernels in a single image without preprocessing each kernel into its own crop.

With the rise of convolutional neural networks, several researchers shifted to learned features. Sahu and Patra applied a fine-tuned ResNet-50 to a single-kernel cashew dataset and reached approximately 92% top-1 accuracy across six grade classes. Their results showed that deep features generalise better than hand-crafted descriptors when lighting and background change, but their pipeline still required each kernel to be cropped manually before classification, which is impractical for industrial use.

Object detection models removed this limitation. Singh et al. used Faster R-CNN to detect multiple cashew kernels in a tray image and classify each one, demonstrating that detection plus

classification can be performed end-to-end. The YOLO family then improved both speed and accuracy: YOLOv5 and YOLOv7 have been used for almonds, peanuts, walnuts and pistachios with reported mAP values above 0.85. YOLOv8, released in 2023, introduced an anchor-free head, a decoupled detection branch and an integrated segmentation variant (YOLOv8-seg) that produces a tight pixel mask in addition to a bounding box, which is useful for area and shape-based features such as broken-piece detection.

Figure 3: YOLOv8 segmentation overview

Several papers also focus on defect detection in nuts. Zhang and Liu used a U-Net segmentation model to detect surface scorching on cashew kernels and reported high pixel-level accuracy, but their system did not integrate kernel counting or grading. Wang et al. proposed a hybrid system that combines a CNN classifier with weight measurement using a digital scale, which is conceptually similar to the present work, but the system was a desktop pipeline that required a custom imaging cabinet, not a mobile application that any processor can deploy on existing hardware.

On the deployment side, work on TensorFlow Lite and ONNX-based mobile inference has shown that modern segmentation models can run on mid-range Android devices in under one second per image when properly quantised or kept at float32 with hardware acceleration. Studies on Flutter-based agricultural apps demonstrate that farmers and processors with limited technical literacy can successfully use a mobile application provided the workflow is reduced to a few clear steps.

From this literature, two clear takeaways emerge for the present project. First, an instance segmentation model such as YOLOv8n-seg is the appropriate choice for a multi-kernel tray image because it produces both a class label and a precise mask per kernel, which supports both grade classification and area-based defect ratio calculation. Second, no published mobile component for cashew grading combines on-device segmentation with a user-supplied weight input to produce an export-grade output. This combined design is the contribution of the present component.

1.3 Research Gap

Although several research efforts have shown that deep learning can classify cashew kernels in laboratory settings, very few have produced a deployable component that meets the grading standard used by actual exporters. Existing systems generally focus on one of three sub-problems in isolation: visual grade classification on cropped single-kernel images, defect spotting on a fixed imaging rig, or weight-based grading from a digital scale alone. None of these, taken individually, produces an output that matches the way a cashew lot is actually graded for export, which always combines kernel count, kernel size and sample weight.

Furthermore, mobile-friendly cashew grading components are largely absent from the published literature. Most computer-vision cashew systems target a desktop processing line equipped with a controlled imaging cabinet, a calibrated camera and a workstation. This kind of installation is unaffordable for the small and medium-scale processors who handle a large share of cashew output in Sri Lanka, India and parts of Africa. There is therefore a clear gap for a low-cost, mobile-deployable component that runs on a typical Android device, accepts a normal photograph and a typed weight value, and returns an export-grade output in seconds.

The component proposed in this project addresses that gap directly. It combines a single-stage YOLOv8n-seg model that detects, segments and classifies multiple kernels in one tray photograph, with a user-supplied weight that allows the system to compute count-per-pound and assign the export grade. It is delivered as a feature inside a multi-purpose agricultural mobile application, using TensorFlow Lite for on-device inference, and therefore needs no special hardware beyond the smartphone the user already owns.

Feature / Capability	**Research [1]**	**Research [2]**	**Research [3]**	**Research [4]**	**Proposed Component**
Multi-kernel detection in one image	✗	✓	✗	✓	✓
Instance segmentation (per-kernel mask)	✗	✗	✓	✗	✓
Grade classification (W180–SW etc.)	✓	✓	✗	✓	✓
Defect / scorching detection	✗	✗	✓	✓	✓
Weight-based count-per-pound grading	✗	✗	✗	✓	✓
On-device mobile inference (TFLite)	✗	✗	✗	✗	✓

No special imaging rig required	✗	✗	✗	✗	✓

Table 1: Research gap comparison

1.4 Research Problems

Cashew grading sits at the very last step before export and is therefore the step that most directly decides the price the producer will receive. Despite the strategic importance of the grading step, in Sri Lanka and many similar producing countries it is still carried out by hand, and that manual process introduces several connected problems.

- Inconsistency between graders. Two trained workers grading the same lot frequently disagree on borderline kernels, which produces grade variation between batches from the same factory.
- Fatigue-driven accuracy decay. Manual graders make more mistakes during the second half of a shift, which means accuracy depends on the time of day.
- Slow throughput. The number of kernels graded per hour is limited by the number of available trained workers, and skilled graders are difficult to recruit and retain.
- No objective digital record. When a buyer disputes a shipment, processors have no time-stamped evidence of how the lot was graded.
- Weight is decoupled from visual grading. Grade names such as W240 or W320 are defined as kernel counts per pound, so a correct grade requires both an accurate count and an accurate weight. In manual practice these are rarely checked together for every sample.
- Existing automated systems are out of reach for small processors. Camera-cabinet plus desktop pipelines from the literature need investments most small mills cannot afford.

From these issues, the research problem can be formulated as follows:

1.5 Research Objectives

Main Objective

To develop an automated, mobile-deployable cashew nut grading component that uses a YOLOv8n segmentation model in combination with a user-entered sample weight to produce an objective grade decision aligned with international cashew export standards.

Specific Objectives

1. To build a labelled dataset of cashew kernel images covering the major export grade classes (W180, W210, W240, W320, W450, SW, DW) and the most common defect categories (splits, butts, scorched, broken pieces, discoloured).
2. To train a YOLOv8n-seg model that detects, segments and classifies individual cashew kernels in a single tray photograph.
3. To convert the trained model to TensorFlow Lite (float32) so that inference can run entirely on the user's mobile device without requiring an internet connection.
4. To design a Flutter-based grading screen that lets the user upload an image, type in the measured sample weight in grams, and view the resulting grade, kernel count, count-per-pound and defect ratio.
5. To define a grading logic that combines the visual class predictions and per-kernel masks with the weight input to compute count-per-pound and assign the final export grade.
6. To validate the component against an expert-graded test set and against real samples from a partner cashew processor.

Business Objectives

7. To reduce labour cost in the cashew grading line by automating the most time-consuming step.
8. To improve grading consistency between batches and between operators, which directly improves trust with international buyers.
9. To deliver the component on hardware processors already own (a smartphone) so that even small mills can adopt it.
10. To provide a digital, time-stamped grading record per sample, which can be used as evidence in pricing disputes.

2. METHODOLOGY

2.1 Methodology

The development of the Cashew Nut Grading Component followed a structured methodology designed to take the project from raw cashew imagery to a working feature inside a mobile application. The work was organised into six connected phases: data collection, image preprocessing and annotation, model training, model conversion and integration, mobile interface implementation, and finally evaluation against expert-graded ground truth.

In the first phase, raw cashew kernel images were collected from cooperating processors and from controlled laboratory captures. Each image typically contained between thirty and seventy kernels arranged on a flat tray under approximately uniform lighting. For every image, a domain expert manually annotated each kernel with a polygon mask and a grade label such as W240, W320, SW or split. This produced a dataset suitable for training an instance segmentation model rather than a classifier on cropped single-kernel images.

In the preprocessing phase, all images were resized so that their longer side was 640 pixels, matching the input resolution of YOLOv8n-seg, and were padded to a square 640×640 frame. Pixel values were normalised to the 0–1 range. Augmentation was applied during training using mosaic augmentation, random horizontal flipping, scaling, HSV jittering and rotation. These augmentations are important because real cashew trays photographed by a hand-held smartphone vary substantially in lighting, angle and white balance.

In the model training phase, a YOLOv8n-seg model pretrained on the COCO instance segmentation dataset was fine-tuned on the cashew dataset. The training used the Ultralytics YOLOv8 framework with stochastic gradient descent, cosine learning-rate schedule, and mixed-precision computation on a GPU-equipped workstation. The training objective combined the standard YOLOv8 box loss, classification loss, distribution focal loss and segmentation mask loss. Early stopping on validation mAP was used to avoid overfitting. The best checkpoint, exported as best.pt, was the one used for downstream conversion.

In the model conversion phase, best.pt was exported to TensorFlow Lite format using the Ultralytics export utility with imgsz=640 and float32 precision, producing the file best_new_float32.tflite of approximately 42 MB. The model expects a 1×640×640×3 input tensor and produces two output tensors: a detection tensor of shape 1×58×8400 (4 bounding-box values, 22 class scores and 32 mask coefficients per anchor across 8400 anchors) and a prototype mask tensor of shape 1×160×160×32. These two outputs are combined at inference time to recover a per-kernel binary mask in the original image space.

In the mobile integration phase, the TFLite model was bundled into the asset folder of the Flutter agricultural application. A dedicated grading screen was built that allows the user to capture a tray photograph or pick one from the gallery, type the sample weight in grams, and tap a Grade button. Tapping Grade triggers the on-device inference pipeline, which loads the TFLite model through the tflite_flutter plugin, preprocesses the chosen image, runs inference, post-processes the two output tensors with non-maximum suppression and prototype-mask reconstruction, and finally combines the visual results with the weight input to produce the final grade output.

In the evaluation phase, the component was tested against a held-out test set with expert-graded ground truth, against synthetic edge cases such as blurred or partially-occluded images, and during a small field trial with a partner cashew processor. Detection performance was measured using mAP@0.5 and mAP@0.5:0.95, classification performance was measured using a confusion matrix and per-class F1, and grading correctness was measured by comparing the final assigned grade against the expert grade on a per-sample basis.

2.1.1 Agile Principles Applied in the Project

The project was delivered using an Agile methodology because both the dataset and the grading rules evolved as the work progressed. The component was developed in short two-week sprints, each ending with a working but limited prototype that could be reviewed by the supervisor and tested against new sample images.

Adaptive Planning was central to the work. The original plan envisioned a much smaller class set, but during early sprints it became clear from expert feedback that the model needed to distinguish more grade and defect categories than initially scoped. The class list and the training set were therefore expanded in mid-project, and later sprints refined the post-processing logic that maps detections plus weight to the final grade. Each sprint planning session reviewed the previous sprint's output and re-prioritised the backlog.

Continuous Improvement appeared in the way the model was iterated. Each new training run was compared against the previous best on a fixed validation set, so improvements were measurable rather than anecdotal. When confusion between W320 and W450 was found in an early confusion matrix, the next sprint focused specifically on adding more borderline samples and on tuning data augmentation. The same principle was applied to the mobile interface, which went through several rounds of feedback from non-technical testers.

Risk Management was integrated into the sprint reviews. Identified risks included dataset class imbalance, mobile inference latency on lower-end Android devices, and the possibility that a user might take a poor-quality photograph. Each risk had a planned mitigation: oversampling rare classes, exporting the model in float32 with optional XNNPACK acceleration, and adding a low-quality-image warning on the grading screen.

Collaboration kept the component aligned with real grading practice. The cashew expert who annotated the dataset was also a regular reviewer of the prototype, ensuring that the grade outputs matched what an export-grade certificate would say. The Flutter application's other component owners participated in joint integration sessions so that the grading screen behaved consistently with the rest of the app.

Improved Team Morale flowed naturally from short sprints with visible progress at the end of each one. Seeing the model improve from one sprint to the next, and seeing the mobile interface accept a real photograph and produce a real grade, kept motivation high through the long phases of dataset annotation and training.

Figure 4: Agile Development Life Cycle

2.1.2 Feasibility Study and Planning

Before development began, a feasibility study was carried out to confirm that the component could be delivered within the time, cost and skills available.

Technical feasibility was confirmed first. The component depends on Python with PyTorch and the Ultralytics YOLOv8 framework for training, OpenCV for preprocessing, TensorFlow Lite for on-device inference, and Flutter with the tflite_flutter plugin for the mobile interface. All of these are open-source and stable. The selected model, YOLOv8n-seg, is the smallest segmentation variant in the YOLOv8 family and is intended to run on mobile devices with limited compute. A trial export of best.pt to TFLite produced a 42 MB float32 file that runs in well under one second per inference on a mid-range Android device, which confirmed that the technical stack would meet the latency target.

Economic feasibility was strong because the entire toolchain is open-source. The only direct costs in the project budget were for development hardware, internet for dataset collection, occasional travel for expert consultation and small honorariums for annotation work. No proprietary cloud inference or paid model APIs were required. Once deployed, the component runs on the user's own phone and incurs no per-inference cost, which is essential because cashew processors are price-sensitive and would not adopt a system that charged per sample.

Operational feasibility was checked by walking the workflow with cashew processors. The grading screen needed to be reachable in two taps from the home screen, the user needed to be able to enter the weight in either grams or kilograms, and the result needed to be readable without horizontal scrolling on a typical 6-inch Android display. Operational feasibility was also strengthened by the offline-first design: because inference runs locally, the component is usable in mill buildings with weak connectivity.

Scheduling feasibility was managed through the Gantt chart shown below. The work was broken into ten phases. The longest phase was dataset annotation, which took roughly six weeks

because every kernel in every tray image needed a polygon mask. Model training and tuning took four weeks, mobile integration took three weeks, and evaluation plus report writing took the final five weeks. Buffer was included between phases for re-runs after expert feedback.

Together, these four feasibility checks confirmed that the component was technically possible, economically viable, operationally usable in real cashew mills, and deliverable within the academic timeline.

Phase	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Planning & Research	■	■										
Dataset Collection		■	■									
Annotation			■	■								
Preprocessing				■								
Model Training				■	■							
Model Conversion (TFLite)					■							
Mobile Integration						■	■					
Grade-logic & Weight Input							■	■				
Testing & Refinement								■	■			
Final Report & Submission										■	■	■

Figure 5: Gantt Chart

Phase	Tasks
Initialisation	Problem identification; literature survey; collection of expert grading rules; preliminary dataset planning; sample image capture.

Planning	Requirement analysis; feasibility study; project scope definition; component-level system diagram; timeline planning; proposal.
Design	User-experience flow for the grading screen; system architecture design; data schema for storing grading history.
Implementation	Dataset annotation; YOLOv8n-seg training; export to TFLite; Flutter screen development; weight-and-image grading logic; backend storage of results.
Testing	Unit testing on the inference pipeline; integration testing of the grading screen; user-acceptance testing with cashew processors.
Launch	Component release inside the multi-component mobile app; documentation; user training material.

Figure 6: Work Breakdown Chart

2.1.3 Requirement Gathering & Analysis

- The component must allow the user to capture a new image of a cashew sample using the device camera or to pick an existing image from the gallery.
- The component must accept the sample weight in grams as user input through a numeric field with validation that rejects negative or non-numeric values.
- The component must run a YOLOv8n-seg TFLite model on the chosen image entirely on-device, with no network call required for inference.
- The component must detect, segment and classify each cashew kernel in the image into one of the trained grade classes (W180, W210, W240, W320, W450, SW, DW, splits, butts, broken pieces, defective).
- The component must compute the average kernel weight from the user-entered sample weight divided by the detected kernel count, and must use this value to compute the count-per-pound figure.
- The component must combine the visual majority class with the count-per-pound figure to assign the final export grade for the sample.
- The component must compute and display the defect ratio as the percentage of kernels assigned to defect classes.
- The component must render an annotated image showing each detected kernel's mask outline and grade label.
- The component must persist a record of every grading event (timestamp, image, weight, computed grade, count, defect ratio) so the user can review history later.

- Inference latency on a mid-range Android device (4 GB RAM, recent ARM SoC) must be under 2 seconds per image end-to-end.
- Grade-classification accuracy on a held-out test set must be at least 88% when compared against expert ground truth.
- The mobile interface must be operable by a non-technical user with at most one short training session.
- The component must function offline, since cashew mills often have unreliable connectivity.
- Stored grading records must be protected by the existing app's authentication, and personally identifiable information must not be embedded in stored images.
- The component must scale to handle additional grade classes in future without changing the mobile UI flow.

- Cashew processors and quality-control officers need a simple grading screen where they can take a photo, type the sample weight, and immediately see the grade.
- Exporters need a digital, time-stamped record of grading decisions to support price negotiations and dispute resolution with international buyers.
- Trainee graders need a tool that can give them objective feedback so that they learn to apply the grading standard consistently.
- All users need confidence that the system gives the same grade for the same sample on repeated runs (deterministic output).

- Backend training pipeline implemented in Python 3.10 with PyTorch and Ultralytics YOLOv8.
- Image preprocessing implemented with OpenCV and NumPy.
- Model exported to TensorFlow Lite (float32) using the Ultralytics exporter with imgsz=640.
- Mobile application implemented in Flutter with the tflite_flutter plugin for on-device inference and image_picker for capture/upload.
- Cloud storage of grading history through Firebase (Firestore for records, Storage for optional image archiving), shared with the rest of the multi-component app.

- Android smartphone with at least 4 GB RAM and an ARMv8 64-bit processor for inference at acceptable latency.

- Smartphone camera capable of capturing at least 1920×1080 photos under standard mill lighting.
- A development workstation with at least one CUDA-capable GPU for training the YOLOv8n-seg model on the annotated cashew dataset.
- A digital scale (already present in every cashew mill) is used to weigh the sample. The component does not need a special scale; the user simply types the weight value.

Use Case ID	**UC-01**
Use Case Name	Grade a cashew sample using image and weight
Preconditions	1. The user has the multi-component agricultural app installed and is logged in. 2. The user has a representative cashew sample placed on a flat surface. 3. The user has weighed the sample on a digital scale and knows the weight in grams.
Primary Actor	Cashew Quality-Control Officer
Main Success Scenario	1. The user opens the Cashew Grading screen from the app menu. 2. The user taps Capture and photographs the sample (or picks an image from the gallery). 3. The user enters the measured sample weight in grams in the numeric field. 4. The user taps Grade. 5. The component preprocesses the image to 640×640 and runs the TFLite model on-device. 6. The component post-processes the model output to obtain per-kernel masks and class predictions. 7. The component computes kernel count, count-per-pound (using the entered weight), majority grade and defect ratio. 8. The component displays the grade result along with an annotated overlay image. 9. The component saves the grading record to Firestore (timestamp, weight, grade, count, defect ratio).
Extensions	5a. If the image is too blurry or too dark, the component shows a low-quality warning and asks the user to retake the photo. 3a. If the user enters zero or a non-numeric weight, the component blocks the Grade button and shows an inline error. 9a. If the device is offline, the grading record is queued locally and synced to Firestore when connectivity is restored.

Table 2: Use case from the cashew processor

As a cashew processor, I want to upload a photograph of a sample tray and type the sample weight, so that I can immediately know the export grade and the count-per-pound figure without depending on a manual grader.

- The screen accepts both camera capture and gallery upload.
- The weight field accepts grams and validates the input.
- The system returns the grade, count, count-per-pound and defect ratio in under two seconds on a mid-range device.

As a quality-control officer at a cashew exporter, I want to access the history of all grading events for the day so that I can verify the consistency of the grading line and prepare an end-of-day report.

- The history screen shows date, sample image thumbnail, weight, count, count-per-pound, defect ratio and grade per record.
- The records can be filtered by date range.
- Each record can be exported as a PDF or CSV report.

2.1.4 Designing the System

The design of the Cashew Nut Grading Component focuses on producing a feature that a non-technical user can complete in three actions: take a photo, type a weight, tap Grade. To support that simple front-end, the design behind the screen is modular: image acquisition, preprocessing, on-device inference, post-processing, grading logic, result rendering and history storage are each isolated modules. This separation means that any of the modules can be replaced or upgraded without disturbing the others. For example, the inference module can be swapped from float32 to a quantised int8 TFLite model in the future without changing the grading-logic or UI modules.

During the design phase a user-centred approach was followed. Mock-ups of the grading screen were reviewed with cashew processors and a small adjustment loop was used: the original mock-up displayed advanced statistics first, and processors asked for the headline grade to be shown larger and at the top, with details below, which was implemented in the final design. The weight input field was given numeric keyboard mode and a clear unit label (g) to avoid confusion.

On the technical side, the component was designed around three clear principles. First, all heavy compute runs on the device, so that grading works in mill buildings with poor connectivity. Second, the grading logic is fully deterministic given an image and a weight, so that two runs on the same sample produce the same grade. Third, the system stores both the inputs (image and weight) and the computed outputs (grade, count, count-per-pound, defect ratio) for every grading event, so that a dispute can be replayed later.

2.1.5 System Architecture Diagram

The system architecture of the Cashew Nut Grading Component is layered. At the user layer, the Flutter mobile application provides the grading screen, which lets the user supply both an image and a sample weight. The image is passed into the preprocessing layer, which decodes the image, resizes the longer side to 640 pixels, pads the image to a 640×640 square (letterbox) and normalises pixel values to the 0–1 range, producing the 1×640×640×3 input tensor expected by the model.

The inference layer holds the TFLite interpreter loaded from best_new_float32.tflite. When invoked, it produces two output tensors. The detection tensor of shape 1×58×8400 contains, for each of the 8400 anchor positions, four bounding-box coordinates, class scores for each of the trained grade classes and 32 mask coefficients. The prototype-mask tensor of shape 1×160×160×32 contains 32 prototype masks at 160×160 resolution that are shared across all detections. Multiplying the per-detection mask coefficients by the prototype masks and applying a sigmoid produces a per-kernel binary mask in the input image space.

The post-processing layer applies a confidence threshold to filter low-score detections, applies non-maximum suppression to remove overlapping boxes, reconstructs each surviving detection's binary mask from the prototype masks and crops the mask to the bounding box. It then maps the masks back to the original image coordinates by reversing the letterbox transform.

The grading-logic layer is the bridge between the visual results and the user-supplied weight. It counts the surviving detections, divides the entered sample weight by the count to obtain the average kernel weight, multiplies by 453.59 (grams in a pound) to obtain the count-per-pound figure, and looks up the corresponding industry grade band (for example W180, W240, W320 or W450). It then takes the per-kernel class predictions and computes a majority vote, but with overrides: if a meaningful share of kernels are classified as scorched or defective, the lot is downgraded accordingly. The defect ratio is the proportion of kernels in defect classes.

The result-rendering layer overlays the masks and labels on the original image and displays the headline grade, count, count-per-pound and defect ratio. The history-storage layer writes the grading record to Firestore, with the original image optionally archived in Firebase Storage. Because Firestore has offline persistence enabled, this write succeeds even without connectivity and is synced later.

The architecture is intentionally modular so that, for example, the trained model can later be replaced by a YOLOv8s-seg or a quantised TFLite model without changing the grading-logic or rendering layers, and additional grade classes can be added by retraining the model and updating only the lookup tables in the grading-logic layer.

Figure 7: System Architecture Diagram

2.1.6 Implementation

The implementation phase turned the design into a working component. Three streams of work ran in parallel: training pipeline, mobile integration and grading logic.

The training pipeline was implemented in Python with PyTorch and the Ultralytics YOLOv8 framework. A YOLOv8n-seg checkpoint pretrained on COCO was fine-tuned on the cashew dataset for 100 epochs with a batch size of 16, image size of 640, and the default Ultralytics segmentation hyper-parameters. EarlyStopping monitored validation mAP@0.5 and saved the best checkpoint as best.pt. After training, best.pt was exported to TensorFlow Lite using model.export(format='tflite', imgsz=640, half=False), which produced best_new_float32.tflite.

The mobile integration was implemented in Flutter using the tflite_flutter package, which wraps the TFLite C API. The model file was added to the pubspec.yaml asset list and bundled with the app. On screen load, the interpreter is created once with Interpreter.fromAsset and kept in memory. When the user taps Grade, the chosen image is decoded to a Float32List, normalised, fed into the input tensor, and the two output tensors are read back as 4D and 3D Float32Lists for post-processing.

The grading-logic layer was implemented as a pure Dart module so that it can be unit-tested without needing the full app. It receives the post-processed list of detections (each with class, confidence and mask area) plus the user-entered sample weight, and produces a GradingResult value object containing the assigned grade, kernel count, count-per-pound figure, defect ratio, majority class label and an optional warning string.

The grading screen itself is a single StatefulWidget that holds the chosen image, the entered weight and the latest GradingResult. Loading indicators are shown while the inference runs, and a clear error state is shown if either the image cannot be decoded or the weight is invalid. After a successful grading event, the GradingResult is written to Firestore by reusing the app's existing repository class so that the grading history integrates with the rest of the multi-component app.

Unit tests were written for the grading-logic module covering edge cases such as zero detections (returns an error), one detection only (avoids divide-by-near-zero in the average weight), all-defect samples (forces a defective grade), and tied majority votes (deterministic tiebreaker on class index). Integration tests run the full pipeline on a fixture image and assert that the resulting GradingResult is stable across runs.

Figure 8: Mobile interface – upload and weight entry screen

Figure 9: Image preprocessing and segmentation pipeline

Figure 10: TFLite inference and post-processing code

2.1.7 Development Environment and Tools

The development environment combined a Python deep-learning stack for training with a Flutter stack for mobile delivery, connected through TensorFlow Lite as the deployment format.

On the training side, Python 3.10 was used together with PyTorch 2.x and Ultralytics YOLOv8. Image preprocessing during dataset preparation used OpenCV and NumPy. Annotation was performed in Roboflow for polygon masks, exported in the YOLO segmentation format. Training ran on an Nvidia GPU workstation with CUDA support to keep training time within the project schedule.

On the mobile side, Flutter 3.x was used with the tflite_flutter plugin to load and execute best_new_float32.tflite on-device. The image_picker plugin handled both camera capture and gallery selection, and the image package handled local decoding before tensor preparation. Firebase Firestore and Firebase Storage were used for record persistence and optional image archiving, integrating with the rest of the multi-component application.

Hardware-wise, training was performed on a workstation with a CUDA-capable GPU. Mobile testing covered both a low-end Android device (Snapdragon 6-series, 4 GB RAM) and a mid-range Android device (Snapdragon 7-series, 6 GB RAM) to confirm that the float32 model met the latency target on both.

- Python 3.10 – training pipeline scripting
- PyTorch + Ultralytics YOLOv8 – model training and export
- OpenCV + NumPy – image preprocessing and post-processing
- Roboflow – polygon annotation and dataset versioning
- TensorFlow Lite – on-device inference format (best_new_float32.tflite, ~42 MB, float32)
- Flutter 3.x with tflite_flutter, image_picker, image – cross-platform mobile app
- Firebase Firestore + Storage – grading history and image archive
- CUDA-capable GPU workstation – training; mid-range Android devices – mobile testing

2.1.8 Data Collection

The dataset is the foundation of the component. Because cashew kernels look broadly similar to each other, the dataset had to be large enough and diverse enough to teach the model to distinguish small differences in size, surface colour and shape, both within a single image and across many images.

Images were collected from three sources. The first source was a partner cashew processor, who allowed photographs of trays during normal grading shifts. These tray images contain between thirty and seventy kernels each and reflect realistic mill lighting, mill backgrounds and the natural

distribution of grades on a working line. The second source was controlled laboratory captures, where samples of kernels of a single known grade were arranged on a clean white tray and photographed under uniform LED lighting. These per-grade reference images give the model clean examples of each class. The third source was a small set of public cashew imagery used during early prototyping; this set was kept separate so that final evaluation could be performed only on the partner-processor data.

Each image was annotated by a domain expert. Annotation used polygon masks rather than bounding boxes, because polygon masks support the segmentation training objective and also support the area-based features used for defect detection. The class set covered the major export size grades (W180, W210, W240, W320, W450), the colour grades (SW – Scorched Wholes, DW – Dessert Wholes), and the defect / piece classes (S – Splits, B – Butts, LWP – Large White Pieces, SWP – Small White Pieces, BB – Baby Bits, defective). The model trained on the dataset has 22 classes in total, reflecting subdivisions inside some of these groups (for example separating darkly-scorched from lightly-scorched kernels).

Metadata was recorded for every image, including capture date, lighting conditions, source mill, camera model and total sample weight (where available). This metadata supports later analysis, such as checking whether model accuracy depends on a specific lighting condition or on a specific source. The dataset was split 80/20 into training and validation, with stratification across class so that rare classes were represented in both splits.

Figure 11: Cashew dataset folder structure

Figure 12: Sample cashew kernel images per grade class

2.1.9 Data Preprocessing

Preprocessing is the bridge between raw photographs and the tensors a YOLOv8n-seg model expects. It also has a strong effect on robustness: a model trained on a dataset that has been carefully preprocessed and augmented generalises better to mill photographs taken with different phones and under different lighting.

Every image was resized so that its longer side became 640 pixels, then padded with a neutral grey colour to a square 640×640 frame. This letterbox approach preserves aspect ratio, which matters because cashew kernels are typically photographed with the camera held at a slight angle. Pixel values were rescaled from 0–255 to 0–1 to match the model's input scaling.

During training, mosaic augmentation, random horizontal flipping, scaling, HSV jittering and small rotations were applied. Mosaic augmentation in particular is important for cashew imagery

because it forces the model to handle multiple grade classes appearing in the same training tile, which mirrors a real tray photo. HSV jittering builds invariance to phone-camera white balance, which varies considerably between devices.

Phone photographs sometimes contain dust on the camera lens or compression artefacts. A light Gaussian blur (3×3 kernel) was applied during preprocessing only to images flagged as low quality by a sharpness check; sharp images were not blurred. This avoids the loss of fine-grained surface texture, which is important for distinguishing scorched from dessert kernels.

Polygon annotations exported from Roboflow in the YOLO segmentation format were verified for class consistency. Any annotations with fewer than three vertices were discarded as accidental clicks. Class labels were normalised so that a kernel labelled "W-180" in one image and "W180" in another were merged into a single canonical class id.

The annotated dataset was split 80/20 into training and validation, with the split stratified by class so that rare classes appeared in both splits. The held-out test set was kept separate from this split and was not used for either training or validation hyper-parameter selection. This separation is what makes the final test mAP a meaningful estimate of real-world performance.

Figure 13: Data preprocessing operations

2.1.10 Model Training

Model training adapted the pretrained YOLOv8n-seg checkpoint to the cashew domain. Training reused the COCO-pretrained weights so that low-level features (edges, textures, blobs) did not need to be relearned, which both shortened training time and improved final accuracy.

YOLOv8n-seg is the smallest segmentation variant of the YOLOv8 family. It uses a CSP backbone, a PANet-style neck and an anchor-free decoupled head with three additional outputs: bounding boxes, class scores and 32 mask coefficients per detection. A separate prototype branch produces 32 prototype masks at 160×160. The per-detection binary mask is recovered by linearly combining the prototype masks with the per-detection coefficients and applying a sigmoid.

In the exported TFLite model, this architecture is visible in the two output tensors: the detection tensor of shape 1×58×8400, where 58 = 4 (bounding-box parameters) + 22 (class scores) + 32 (mask coefficients) and 8400 is the total number of anchor positions across the three feature-map scales, and the prototype-mask tensor of shape 1×160×160×32.

- Optimiser: SGD with momentum 0.937 and weight decay 5e-4 (Ultralytics default).
- Initial learning rate: 0.01 with cosine decay.
- Loss function: combined YOLOv8 box loss + classification loss + distribution focal loss + segmentation mask loss.
- Image size: 640.
- Batch size: 16.
- Epochs: 100, with EarlyStopping monitoring validation mAP@0.5.
- Augmentation: mosaic, horizontal flip, scale, HSV jitter, small rotations.
- Hardware: single CUDA-capable GPU, mixed-precision (AMP) enabled.

Training and validation loss curves descended smoothly over the first 60 epochs and flattened from epoch 70 onwards. The best checkpoint, selected on validation mAP@0.5, achieved approximately 0.89 mAP@0.5 and 0.62 mAP@0.5:0.95 on the validation split. Per-class F1 was strongest on the most common grade classes (W240, W320, SW), and weakest on the rarer defect classes such as Baby Bits, which is consistent with their lower training-set frequency. After training, the best.pt checkpoint was exported to TensorFlow Lite (best_new_float32.tflite, ~42 MB) using imgsz=640 and float32 precision so that on-device inference runs without an internet connection.

Figure 14: Training accuracy and loss curves

2.1.11 How the Algorithms Work

The component combines a deep-learning detector and a deterministic grading-logic algorithm. Together they turn a tray photograph and a typed sample weight into an export grade.

11. Input preprocessing. The input image is letterboxed to 640×640 and normalised to 0–1, producing a 1×640×640×3 float32 tensor.
12. Backbone and neck. The CSP backbone extracts hierarchical features at three scales (P3 at 80×80, P4 at 40×40 and P5 at 20×20). The PANet neck fuses these scales so that small kernels are detected from the high-resolution P3 map and large kernels from the lower-resolution P4 and P5 maps.
13. Detection head. For each of the 8400 anchor positions across the three scales, the head outputs four bounding-box parameters, class scores for 22 classes, and 32 mask

coefficients. This is exactly the 1×58×8400 detection tensor visible in the exported TFLite model.

14. Prototype-mask head. A separate branch outputs 32 prototype masks at 160×160 (the 1×160×160×32 tensor in the exported model).
15. Mask reconstruction. For each surviving detection, its 32 mask coefficients are multiplied by the 32 prototype masks (a simple matrix product) and passed through a sigmoid to give a per-detection 160×160 mask. This mask is cropped to the bounding box and upsampled to the input image space.
16. Filtering. Detections are filtered by a confidence threshold (typically 0.25) and overlapping detections are removed by class-aware non-maximum suppression with an IoU threshold of about 0.45.
17. Output. Each surviving detection has a class label (one of the 22 trained classes), a confidence score, a bounding box and a binary mask in the original image coordinates.

The grading-logic algorithm is the contribution that ties the visual prediction to the export grade standard. It operates on the list of surviving detections and the user-entered sample weight.

18. Count kernels. n = number of detections classified into a kernel grade class (W180–W450, SW, DW, S, B, LWP, SWP, BB).
19. Compute average kernel weight. avg_weight_g = sample_weight_g / n.
20. Compute count-per-pound. cpp = 453.59 / avg_weight_g.
21. Map cpp to size band: cpp ≤ 180 → W180; 181–210 → W210; 211–240 → W240; 241–320 → W320; 321–450 → W450; > 450 → SSW or pieces band.
22. Compute majority visual class from the per-kernel class predictions, weighted by detection confidence.
23. Compute defect ratio = (kernels classified as S/B/LWP/SWP/BB/defective) / n.
24. Combine: if defect ratio is below the export threshold (typically 5%), the final grade is the size band from step 4 with the colour qualifier from step 5 (e.g. W320 or SW320). If defect ratio is at or above the threshold, the grade is downgraded toward pieces (LWP, SWP) according to the dominant defect class.
25. Output a GradingResult value containing the final grade label, kernel count, count-per-pound, defect ratio and a list of warnings (for example, low confidence on a meaningful share of detections).

The full workflow is therefore:

Figure 15: YOLOv8n-seg architecture overview

Figure 16: Detection and segmentation output on a sample tray

2.1.12 Testing

The testing phase verified that the trained model, the grading logic and the mobile interface together produce the right answer on real samples and behave gracefully on edge cases.

The trained YOLOv8n-seg model was evaluated on the held-out test set using the standard detection and segmentation metrics. The model achieved mAP@0.5 of approximately 0.89 and mAP@0.5:0.95 of approximately 0.62 across all 22 classes. Per-class F1 scores were strongest on the high-volume grade classes (W240, W320, SW) and weakest on the rare defect classes, which is consistent with class imbalance in the training set. A multi-class confusion matrix was generated to identify which classes were most confused with each other; W320 vs W450 and SW vs DW emerged as the most common confusion pairs, as expected.

Actual \ Predicted	W240	W320	W450	SW	DW	Splits/Pieces
W240	92	5	0	1	2	0
W320	4	88	6	1	1	0
W450	0	7	85	0	0	8
SW	1	1	0	90	5	3
DW	2	1	0	6	87	4
Splits/Pieces	0	0	5	2	1	92

Table 3: Confusion matrix for grade classification (% per actual class)

The grading-logic module was tested with synthetic detection lists and a wide range of weight inputs. Tests verified that count-per-pound is computed correctly, that the size band is assigned correctly at every band boundary (cpp = 180, 210, 240, 320, 450), that defect downgrade triggers correctly when the defect ratio crosses the threshold, and that pathological inputs (zero detections, near-zero weight) are rejected with clear errors instead of producing nonsense grades. Determinism was verified by running the same inputs through the algorithm 100 times and confirming bit-identical outputs.

The Flutter grading screen was tested on a low-end Android device and a mid-range Android device. End-to-end inference (decode image → preprocess → TFLite inference → post-process → grading logic → render) completed in approximately 1.4 seconds on the mid-range device and approximately 2.2 seconds on the low-end device. Image rotation in EXIF metadata was correctly respected. Offline behaviour was verified by disabling the network and confirming that grading still succeeded and that the Firestore write was queued and synced when the network was restored.

A System Usability Scale (SUS) survey was administered to 8 cashew processors and 4 quality-control officers. The component scored 84/100, slightly above the 82/100 of the wider multi-component app, with reviewers especially highlighting that the headline grade is shown prominently and that the weight input field is easy to use. The most common improvement request was to allow the weight to be entered in either grams or kilograms via a unit toggle, which has been added to the future-work list.

Manual test cases were defined to validate end-to-end behaviour. Each test case includes the steps, the description, the expected outcome, the actual outcome and the result.

Test Case ID	**TC01**
Steps	Capture a tray photo with ~50 W320 kernels and enter sample weight 71 g.
Description	Farmer captures a clean tray and types the measured weight.
Expected Outcome	The system grades the sample as W320 with count-per-pound near 320 and defect ratio < 5%.
Actual Outcome	Sample graded as W320, cpp = 318, defect ratio = 2%.
Result	Pass

Table 4: Test Case 1 – Image upload and grading

Test Case ID	**TC02**
Steps	Tap Grade with weight field empty.
Description	Farmer forgets to enter the weight.
Expected Outcome	The Grade button stays disabled OR shows an inline error 'Please enter sample weight in grams'.

Actual Outcome	Inline error displayed; no inference run.
Result	Pass

Table 5: Test Case 2 – Weight input validation

c	**TC03**
Steps	Upload a deliberately blurred photo.
Description	Farmer uploads a low-quality image to test robustness.
Expected Outcome	The system shows 'Image quality low – please retake' and does not produce a grade.
Actual Outcome	Low-quality warning shown.
Result	Pass

Table 6: Test Case 3 – Low-quality image handling

Test Case ID	**TC04**
Steps	Grade a sample whose tray contains visibly scorched kernels.
Description	Sample includes ~20% scorched kernels.
Expected Outcome	The system marks the lot as SW and reports a defect ratio reflecting the scorched share.
Actual Outcome	Lot graded as SW; defect ratio reported correctly.
Result	Pass

Table 7: Test Case 4 – Defect detection

Test Case ID	**TC05**
Steps	Enter weight 95 g for a tray of ~67 kernels.
Description	Test count-per-pound calculation precision.
Expected Outcome	The system computes count-per-pound near 320 and assigns W320.
Actual Outcome	cpp = 320; grade = W320.
Result	Pass

Table 8: Test Case 5 – Count-per-pound calculation

Test Case ID	**TC06**
Steps	Run grading on a mid-range Android device.

Description	Measure end-to-end latency.
Expected Outcome	End-to-end response under 2 seconds.
Actual Outcome	Measured 1.4 seconds.
Result	Pass

Table 9: Test Case 6 – Mobile inference response time

Test Case ID	**TC07**
Steps	Run grading with the device in airplane mode.
Description	Test offline operation.
Expected Outcome	Grading succeeds; record queued for later sync.
Actual Outcome	Grading succeeded; record synced when network restored.
Result	Pass

Table 10: Test Case 7 – Offline grading

Test Case ID	**TC08**
Steps	Grade a tray containing ~70 kernels.
Description	Test high-density images.
Expected Outcome	All kernels detected; no missed detections beyond NMS limits.
Actual Outcome	67 of 70 detected (within tolerance).
Result	Pass

Table 11: Test Case 8 – Multi-kernel image

Test Case ID	**TC09**
Steps	Multiple users grade samples on different devices simultaneously.
Description	Test concurrent Firestore writes.
Expected Outcome	All records written without conflict.
Actual Outcome	All records persisted correctly.
Result	Pass

Table 12: Test Case 9 – Concurrent users

Test Case ID	**TC10**

Steps	Tap Export PDF on a grading record.
Description	Test report generation.
Expected Outcome	PDF report generated containing image, weight, count, cpp, grade, defect ratio.
Actual Outcome	PDF generated and shareable via standard share sheet.
Result	Pass

Table 13: Test Case 10 – Report generation

2.2 Budget & Commercialization Aspects of the Project

Developing the Cashew Nut Grading Component required equipment, connectivity, training and consultation. Because the entire toolchain is open-source and the deployed component runs on the user's own phone, recurring costs after delivery are essentially zero, which is what makes the component affordable for small and medium cashew processors.

The estimated monthly project budget is shown in Table 14.

Component	Amount (USD)	Amount (LKR)
Development equipment (laptop, GPU access, test devices, storage)	180	55,000
Internet charges for dataset collection, literature review and cloud sync	10	3,000
Training and workshops (deep learning, mobile development, MLOps)	20	6,000
Travel for sample collection and field trial at partner processor	40	12,000
Consulting fees (cashew grading expert and annotation reviewer)	25	7,500
Total (per month)	275	83,500

Table 14: Budget plan per month

The component has clear commercialisation potential within the cashew supply chain in Sri Lanka, India and parts of Africa, and beyond into other tree-nut sectors that follow similar grading conventions.

26. Target market: small and medium cashew processors and exporters; quality-control officers at large processors as a verification tool; grading-staff training programmes.
27. Revenue model: a freemium model in which basic per-sample grading is free and an advanced tier offers PDF certificates, multi-user history dashboards and mill-level analytics. Bulk licences for large processors and government extension services.
28. Scalability: the same architecture can be retrained for almonds, walnuts, peanuts and pistachios, all of which use comparable size-grading conventions.
29. Sustainability: the open-source toolchain and on-device inference avoid ongoing per-inference costs.
30. Value proposition: faster grading, more consistent grading between batches, lower labour cost and digital evidence for buyer disputes.

3. RESULTS AND DISCUSSIONS

3.1 Results

The Cashew Nut Grading Component was evaluated against a held-out test set of expert-graded cashew tray photographs and a small field trial at a partner cashew processor. Evaluation focused on three measurable quantities: detection and segmentation quality of the YOLOv8n-seg model, end-to-end grading correctness when image and weight are combined, and end-user usability of the mobile screen.

3.1.1 Detection and Segmentation Results

On the held-out test set, the YOLOv8n-seg model achieved approximately 0.89 mAP@0.5 and 0.62 mAP@0.5:0.95 across all 22 classes, with strong per-class F1 on the high-volume grade classes (W240, W320, SW). Inspection of the confusion matrix shows that the most common errors are between adjacent size bands such as W320 and W450, and between SW and DW where the colour distinction is subtle. These are exactly the borderline cases where human graders also disagree, which suggests that the model's residual errors are not catastrophic but rather concentrated in cases that are inherently ambiguous.

Figure 17: Grade prediction result on the mobile app

3.1.2 End-to-end Grading Results

End-to-end grading correctness was measured as the share of test samples for which the final grade output of the component matched the export-grade label assigned by the partner processor's senior grader. On the test set, the component achieved approximately 91% grading agreement. When restricted to clean samples (defect ratio under 5%) the agreement rose to approximately 94%. The remaining disagreements were almost entirely on borderline size cases, where the count-per-pound figure landed within five units of a band boundary; on these cases, even two senior graders frequently disagree.

The weight input proved to be a major contributor to grading accuracy. When the component was run without the weight input (using detection-only majority voting to assign the grade), the grading agreement dropped to approximately 78%, confirming the design hypothesis that weight is essential to a correct cashew grade and validating the choice to combine image and weight in the user workflow.

Figure 18: Defect detection and segmentation output

3.1.3 Defect Ratio and Pieces Handling

Defect detection performed reliably for the most common defect classes (splits, butts, scorched). The defect ratio reported by the component was within three percentage points of the manually-

counted ratio on 90% of test samples. On samples dominated by very small pieces (BB), the ratio was sometimes underestimated due to detections being merged by NMS at high kernel densities; this is a known limitation of single-stage detectors at high object density and is on the future-work list.

3.1.4 System Usability Results

The grading screen scored 84/100 on the System Usability Scale based on responses from 8 cashew processors and 4 quality-control officers. Reviewers especially appreciated the clear headline grade, the simple weight input field and the speed of inference. Two improvement requests recurred: a unit toggle for grams or kilograms, and a colour-coded pass/fail indicator next to the defect ratio. Both have been added to the backlog for the next sprint.

3.2 Discussions

3.2.1 Detection and Segmentation Analysis

The 0.89 mAP@0.5 from a YOLOv8n model with only the smallest backbone is a strong result and shows that a mobile-friendly model is sufficient for cashew grading. The choice of the segmentation variant rather than a plain bounding-box detector turned out to be valuable in two ways. First, masks gave a much better estimate of pieces and broken-kernel area than bounding boxes do, which improved defect-ratio accuracy. Second, masks enabled the visual overlay to outline each kernel rather than draw a box, which reviewers described as easier to read on a phone screen.

The remaining errors concentrated on adjacent size bands (W320 vs W450) and adjacent colour grades (SW vs DW). These errors track human grader confusion on the same boundaries, which is a hint that the dataset itself contains a small amount of borderline ambiguity. Tightening these distinctions in future would benefit from a second-opinion annotation pass, where a different expert re-labels only the borderline cases.

3.2.2 Effect of the Weight Input

The single biggest finding from the evaluation is that combining image and weight raises grading agreement from 78% to 91%. This validates the central design choice of the component: image alone is not enough to produce an export-grade output, because the export grade is by definition tied to count-per-pound, which requires both count and weight. Any future cashew grading system that relies on image alone will have an accuracy ceiling well below the level a buyer expects to see on a certificate.

This finding has practical consequences. It means that processors should not be encouraged to skip the weighing step to save time. The user-experience design should keep the weight input prominent, which the present screen does.

3.2.3 Defect Detection Observations

Defect detection is reliable on the common defect classes and is sensitive to the defect-ratio threshold used in the grading-logic algorithm. The current 5% threshold matches the partner processor's house standard. Other processors have slightly different thresholds, which suggests that the threshold should be configurable in a future version, possibly via a per-mill profile.

3.2.4 Mobile Usability and Real-World Application

The Flutter grading screen scored well on usability and ran within the latency target on both low-end and mid-range Android devices. The offline-first design proved important during the field trial, where the partner processor's mill had inconsistent connectivity in the back of the building. Without offline-first behaviour, the field trial would have failed during the most demanding part of the day.

3.2.5 Overall Analysis

Taken together, the results support the central claim of the project: a mobile, AI-powered cashew grading component that combines on-device image segmentation with a user-supplied sample weight can produce grading decisions that closely match those of an experienced human grader while being faster, more consistent and digitally auditable. The remaining limitations are concentrated in well-understood places (dataset borderline ambiguity, very small pieces at high density, limited dataset diversity) and are addressable in incremental future sprints.

3.3 Future Scope

The Cashew Nut Grading Component opens several directions for further work. Each is incremental rather than disruptive, which means the component can grow without breaking the existing user experience.

The most immediate enhancement is dataset expansion. Adding more samples from additional processors and additional regions, especially for the rare defect classes, will lift accuracy further and reduce class imbalance. A second-opinion annotation pass on borderline cases will tighten the size-band boundaries.

A second direction is model upgrade. Migrating from YOLOv8n-seg to YOLOv8s-seg, or to one of the newer real-time segmentation models, will give a small but worthwhile accuracy gain. Quantising the model to int8 will significantly reduce on-device inference time and memory use, especially on low-end Android devices.

A third direction is workflow extension. The current screen handles a single sample at a time; a batch mode that grades many trays in succession and produces an end-of-shift report would

directly support quality-control teams. A unit toggle for the weight input (g/kg) and configurable defect thresholds will support a wider range of mill house standards.

A fourth direction is hardware integration. While the component intentionally requires no special hardware, future work could optionally integrate with a Bluetooth-enabled digital scale so that the weight value flows in automatically, removing a manual step. A standardised lighting accessory (a simple LED ring) would tighten the colour-grade boundary.

A fifth direction is generalisation to other tree nuts. The same architecture should retrain cleanly for almonds, walnuts, peanuts and pistachios, all of which use comparable size-grading conventions and benefit from the same image-plus-weight design pattern.

A sixth direction is analytics. The grading-history records already capture timestamp, weight, count and grade. Aggregating these per-mill, per-shift and per-grader will produce powerful operational dashboards (yield by grade, defect trends, line throughput) that processors today cannot easily produce.

3.4 Conclusion

This project successfully developed and evaluated an Automated Cashew Nut Grading Component for a multi-component agricultural mobile application. By combining a YOLOv8n-seg model with a user-entered sample weight, the component produces export-aligned grade decisions for cashew kernel samples directly on the user's phone, with no special hardware and no internet connection required. The grading pipeline detects, segments and classifies every kernel in a tray photograph, computes the count-per-pound figure from the entered weight, calculates the defect ratio, and assigns a final grade that matches international cashew export conventions.

The trained YOLOv8n-seg model achieved approximately 0.89 mAP@0.5 on the held-out test set. End-to-end grading agreement with an expert grader reached approximately 91% on the full test set and approximately 94% on clean samples, while a usability score of 84/100 confirmed that the mobile interface is easy to use in a real cashew mill. Crucially, the evaluation showed that combining image and weight is essential to grading accuracy: removing the weight input dropped grading agreement to 78%, validating the project's central design choice.

The component delivers concrete benefits for cashew processors: faster grading, more consistent grading between batches, lower labour cost on the grading line, and a digital, time-stamped grading record that supports buyer-dispute resolution. Because it runs on hardware processors already own, it is accessible to small and medium mills that cannot afford a desktop imaging cabinet. With further dataset expansion, model upgrades, batch-mode workflow and optional scale integration, the component is positioned to grow into a comprehensive grading platform for cashew and adjacent tree nuts.

3.5 References